

Vectors and Matrices

Vectors

A **coordinate system** is defined by an origin point and the orientation and scale of a set of coordinate axes.

A **point** $p = (x, y)$ refers to a location in space, specified relative to a coordinate system.

A **vector** $v = \langle m, n \rangle$ refers to a **displacement**—an amount of change in each coordinate—and is typically drawn as an arrow pointing along the direction of displacement. The figure below shows three (3) vectors:

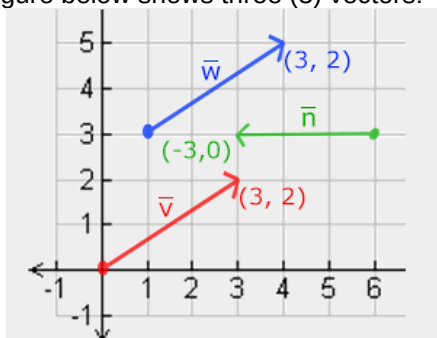


Figure 1. A collection of vectors.

- **Initial point** or **tail** – the point where the arrow begins
- **Terminal point** or **head** – the point where the arrow ends; indicates the result when the displacement has been applied to the initial point.
- **Length** or **magnitude** – the distance between the initial and terminal points of the vector; can be calculated from the components of the vector.

Vectors are not associated with any particular location in space; the same vector may exist at different locations. A vector whose initial point is located at the origin (when a coordinate system is specified) is said to be in **standard position**.

Individual numbers (that are not part of a point or vector) are often called **scalars**.

A variety of mathematical operations can be performed with and upon vectors. The first operation is the addition of vectors. Given vectors $v = \langle v_1, v_2 \rangle$ and $w = \langle w_1, w_2 \rangle$, the formula shall be:

$$v + w = \langle v_1 + w_1, v_2 + w_2 \rangle$$

Sample Vector Addition:

$$v = \langle 2, 5 \rangle$$

$$w = \langle 4, -2 \rangle$$

$$v + w = \langle v_1 + w_1, v_2 + w_2 \rangle$$

$$v + w = \langle 2 + 4, 5 + -2 \rangle$$

$$v + w = \langle 6, 3 \rangle$$

The result is a new vector $u = v + w$ that shares the initial point of v and the terminal point of w .

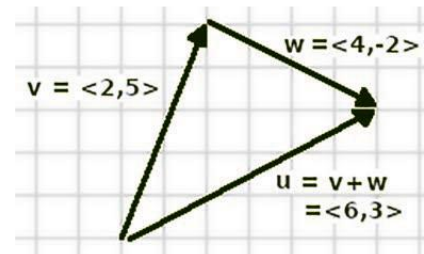


Figure 2. Vector addition.

Vector subtraction is the same as addition with a negated second vector:

$$v = \langle 1, 2 \rangle$$

$$w = \langle 4, 5 \rangle$$

$$v + -w = \langle v_1 + (-w_1), v_2 + (-w_2) \rangle$$

$$v + -w = \langle 1 + (-4), 2 + (-5) \rangle$$

$$v + -w = \langle -3, -3 \rangle$$

Given the length of the two (2) sides (x , y), you can determine the length/magnitude of a vector using the **Pythagorean theorem**.

$$||v|| = \sqrt{x^2 + y^2}$$

For example, the following computes the length of $v = \langle 4, 2 \rangle$:

$$||v|| = \sqrt{4^2 + 2^2} = \sqrt{16 + 4} = \sqrt{20} = 4.47$$

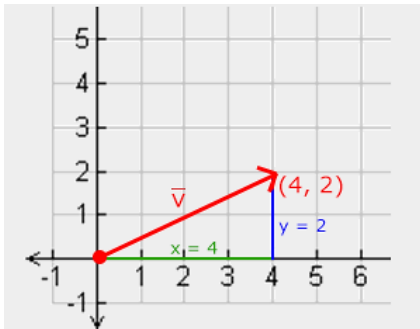


Figure 3. Sample graph showing a vector's length.

Matrices

A **matrix** is a rectangular array of values called **elements** that are typically accessed by means of **subscripts**. The first subscript refers to the row number, while the second one refers to the column number, with the subscripts starting at 0. The figure below shows an example of a 4x4 matrix.

$$\begin{bmatrix} A_{00} & A_{01} & A_{02} & A_{03} \\ A_{10} & A_{11} & A_{12} & A_{13} \\ A_{20} & A_{21} & A_{22} & A_{23} \\ A_{30} & A_{31} & A_{32} & A_{33} \end{bmatrix}$$

Figure 4. A 4x4 matrix.

The **identity matrix** contains all zeros, with ones along the diagonal. Any point or matrix multiplied by the identity matrix is unchanged.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figure 5. A 4x4 identity matrix.

The **transpose** of a matrix is computed by interchanging its rows and columns.

$$\begin{bmatrix} A_{00} & A_{01} & A_{02} & A_{03} \\ A_{10} & A_{11} & A_{12} & A_{13} \\ A_{20} & A_{21} & A_{22} & A_{23} \\ A_{30} & A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{10} & A_{20} & A_{30} \\ A_{01} & A_{11} & A_{21} & A_{31} \\ A_{02} & A_{12} & A_{22} & A_{32} \\ A_{03} & A_{13} & A_{23} & A_{33} \end{bmatrix}$$

Figure 6. A transpose of a 4x4 matrix.

Matrices can be added or subtracted only when they have the same number of dimensions. You can find the sum simply by adding the corresponding entries in matrices.

$$\begin{aligned} A + B &= \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \\ &= \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} \\ &= \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix} \end{aligned}$$

The same rule applies to matrix subtraction:

$$\begin{aligned} A - B &= \begin{bmatrix} 4 & 2 \\ 1 & 6 \end{bmatrix} - \begin{bmatrix} 2 & 4 \\ 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 4-2 & 2-4 \\ 1-0 & 6-1 \end{bmatrix} \\ &= \begin{bmatrix} 2 & -2 \\ 1 & 5 \end{bmatrix} \end{aligned}$$

You can multiply a scalar by each element of the matrix using the dot operator. This is known as **scalar multiplication**.

$$2 \cdot \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 2 \cdot 1 & 2 \cdot 2 \\ 2 \cdot 3 & 2 \cdot 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}$$

In **matrix multiplication**, you can only multiply matrices if the number of columns on the left-hand side matrix is equal to the number of rows on the right-hand side matrix. To multiply matrices, multiply the elements of each row of the first matrix by the elements of each column in the second matrix. Then, add the products.

$$\begin{aligned} A \cdot B &= \begin{bmatrix} 3 & 2 & 1 & 5 \\ 9 & 1 & 3 & 0 \end{bmatrix} \cdot \begin{bmatrix} 2 & 9 & 0 \\ 1 & 3 & 5 \\ 2 & 4 & 7 \\ 8 & 1 & 5 \end{bmatrix} \\ A \cdot B &= \begin{bmatrix} 6+2+2+40 & 27+6+4+5 & 0+10+7+25 \\ 18+1+6+0 & 81+3+12+0 & 0+5+21+0 \end{bmatrix} \\ &= \begin{bmatrix} 50 & 42 & 42 \\ 25 & 96 & 26 \end{bmatrix} \end{aligned}$$

References:

Korites, B. (2018). *Python graphics: A reference for creating 2D and 3D images*. Apress.
 Marschner, S. & Shirley, P. (2021). *Fundamentals of computer graphics* (5th ed.). CRC Press.
 Stemkoski, L. & Pascale, M. (2021). *Developing graphics frameworks with Python and OpenGL*. CRC Press.

Geometric Transformations

Scaling, Rotation, and Translation

A **geometric transformation** refers to a change applied to an object in terms of size, orientation, or position.

The following are the three (3) basic geometric transformations in computer graphics:

1. **Scaling:** The object's dimensions are either expanded or compressed to change its size. A scale matrix can also be used to move points towards or away from the origin. By scaling relative to the origin, all coordinates of the points defining an entity are multiplied by the same factor.
2. **Translation:** The object is moved to another position or location on the screen. All coordinates of the points defining an entity are modified by adding the same vector quantity.
3. **Rotation:** The object is moved around a fixed point at a given angle. In 2D, a rotation transformation rotates vectors around the origin point. In 3D, rotations are performed around a line rather than a point.

Scaling

The **scaling factor** is used to determine whether the size of an object will be increased or reduced. A scaling factor greater than 1 implies that the object's size will be increased. If the scaling factor is less than 1, the object size will be reduced.

Given the initial x, y, z coordinates of an object and the scaling factor for each axis (S_x , S_y , S_z), you can determine the new coordinates by creating a 4x4 matrix. Align the scaling factors diagonally starting from the first element, then perform matrix multiplication.

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix}$$

If the scaling factor for each axis is 1, the scale matrix shall consist of an identity matrix. Any point or matrix multiplied by the identity matrix is unchanged.

Sample Problem:

The initial coordinates of an object are A(3, 3, 6), B(0, 3, 3), C(3, 0, 1), D(0, 0, 0). Determine the object's new coordinates after applying the scaling factors 2 towards each x axis and 3 towards each of the y and z axes.

Solution for A(3, 3, 6):

$$\begin{aligned} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} &= \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 3 \\ 6 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 6 + 0 + 0 + 0 \\ 0 + 9 + 0 + 0 \\ 0 + 0 + 18 + 0 \\ 0 + 0 + 0 + 1 \end{bmatrix} \\ &= \begin{bmatrix} 6 \\ 9 \\ 18 \\ 1 \end{bmatrix} \end{aligned}$$

Solution for B(0, 3, 3):

$$\begin{aligned} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} &= \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 3 \\ 3 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 + 0 + 0 + 0 \\ 0 + 9 + 0 + 0 \\ 0 + 0 + 9 + 0 \\ 0 + 0 + 0 + 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ 9 \\ 9 \\ 1 \end{bmatrix} \end{aligned}$$

Solution for C(3, 0, 1):

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 6 + 0 + 0 + 0 \\ 0 + 0 + 0 + 0 \\ 0 + 0 + 3 + 0 \\ 0 + 0 + 0 + 1 \end{bmatrix}$$

$$= \begin{bmatrix} 6 \\ 0 \\ 3 \\ 1 \end{bmatrix}$$

Solution for D(0, 0, 0):

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 + 0 + 0 + 0 \\ 0 + 0 + 0 + 0 \\ 0 + 0 + 0 + 0 \\ 0 + 0 + 0 + 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

The new coordinates of the object after scaling are A(6, 9, 18), B(0, 9, 9), C(6, 0, 3), D(0, 0, 0).

Translation

A **translation vector**, also called a shift vector, defines the distance to move an object's coordinate. You can determine the new coordinates of an object after translation by creating a 4x4 matrix. Align values of 1 diagonally starting from the first element and set the translation vectors (T_x , T_y , T_z) for the x, y, z axes as the values of the final column. Then, multiply by the initial coordinates of the object.

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix}$$

Sample Problem:

The initial coordinates of an object are A(3, 3, 2), B(0, 3, 1), C(3, 0, 0), D(0, 0, 0). Determine the object's new coordinates after applying the translation vectors 1 towards each of the x and y axes and 2 towards each z axis.

Solution for A(3, 3, 2):

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 3 \\ 2 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 3 + 0 + 0 + 1 \\ 0 + 3 + 0 + 1 \\ 0 + 0 + 2 + 2 \\ 0 + 0 + 0 + 1 \end{bmatrix}$$

$$= \begin{bmatrix} 4 \\ 4 \\ 4 \\ 1 \end{bmatrix}$$

Solution for B(0, 3, 1):

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 3 \\ 1 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 + 0 + 0 + 1 \\ 0 + 3 + 0 + 1 \\ 0 + 0 + 1 + 2 \\ 0 + 0 + 0 + 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ 4 \\ 3 \\ 1 \end{bmatrix}$$

Solution for C(3, 0, 0):

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 3 + 0 + 0 + 1 \\ 0 + 0 + 0 + 1 \\ 0 + 0 + 0 + 2 \\ 0 + 0 + 0 + 1 \end{bmatrix}$$

$$= \begin{bmatrix} 4 \\ 1 \\ 2 \\ 1 \end{bmatrix}$$

Solution for D(0, 0, 0):

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 + 0 + 0 + 1 \\ 0 + 0 + 0 + 1 \\ 0 + 0 + 0 + 2 \\ 0 + 0 + 0 + 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ 1 \\ 2 \\ 1 \end{bmatrix}$$

The new coordinates of the object after translation are A(4, 4, 4), B(1, 4, 3), C(4, 1, 2), D(0, 0, 0).

Rotation

In 3D, there are three (3) types of rotation: x-axis rotation, y-axis rotation, and z-axis rotation. Given the initial x, y, z coordinates of an object and the rotation angle, you can determine the new coordinates by creating 4x4 matrices with combinations of the **sine** and **cosine** functions.

For x-axis rotation:

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix}$$

For y-axis rotation:

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix}$$

For z-axis rotation:

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix}$$

Sample Problem:

Given the initial coordinates (1, 2, 3), determine the new coordinates after applying a rotation of 90 degrees towards the x, y, z axes.

Solution for x-axis rotation:

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos 90^\circ & -\sin 90^\circ & 0 \\ 0 & \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ 3 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ 3 \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 + 0 + 0 + 0 \\ 0 + 0 + -3 + 0 \\ 0 + 2 + 0 + 0 \\ 0 + 0 + 0 + 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ -3 \\ 2 \\ 1 \end{bmatrix}$$

Solution for y-axis rotation:

$$\begin{aligned} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} &= \begin{bmatrix} \cos 90^\circ & 0 & \sin 90^\circ & 0 \\ 0 & 1 & 0 & 0 \\ -\sin 90^\circ & 0 & \cos 90^\circ & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ 3 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 + 0 + 3 + 0 \\ 0 + 2 + 0 + 0 \\ -1 + 0 + 0 + 0 \\ 0 + 0 + 0 + 1 \end{bmatrix} \\ &= \begin{bmatrix} 3 \\ 2 \\ -1 \\ 1 \end{bmatrix} \end{aligned}$$

Solution for z-axis rotation:

$$\begin{aligned} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} &= \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ & 0 & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_i \\ y_i \\ z_i \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ 3 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 + -2 + 0 + 0 \\ 1 + 0 + 0 + 0 \\ 0 + 0 + 3 + 0 \\ 0 + 0 + 0 + 1 \end{bmatrix} \end{aligned}$$

$$= \begin{bmatrix} -2 \\ 1 \\ 3 \\ 1 \end{bmatrix}$$

Projections

In a perspective projection, the shape of the viewable region is called a **frustum** or truncated pyramid. The pyramid is oriented so that its central axis is aligned with the negative z-axis, while the viewer or virtual camera is positioned at the origin of the scene, which aligns with the point that was the tip of the original pyramid.

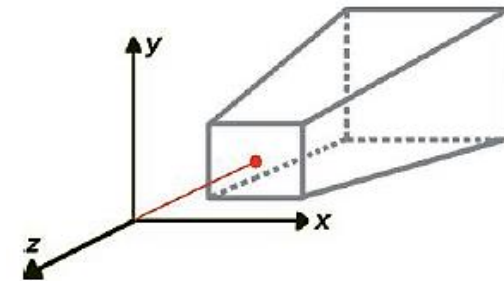


Figure 1. The frustum for a perspective transformation

The smaller rectangular end of the frustum is nearest to the origin, while the larger rectangular end is farthest from the origin. When the frustum is compressed into a cube, the larger end must be compressed more. This causes objects in the rendered image of the scene to appear smaller the farther they are from the viewer.

The shape of a frustum is defined by four (4) parameters. These are near distance, far distance, (vertical) angle of view, and aspect ratio. The **near and far distances** refer to distances from the viewer (along the z-axis). These two set absolute bounds on what could potentially be seen by the viewer. Any points beyond this range will not be rendered. However, not everything between these bounds will be visible. The **angle of view** measures how much of the scene is visible to the viewer. It is defined as the angle between the top and bottom planes of the frustum (as oriented in Figure 1) if those planes were extended to the origin. Figure 2 shows two (2) different frustums (shaded regions) as viewed from the side (along the x-axis). For fixed near and far distances, larger angles of view correspond to larger frustums.

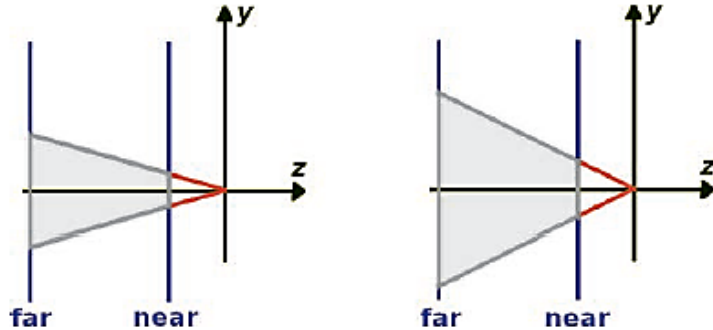


Figure 2. The effect of the angle of view on the size of a frustum

For the dimensions of the visible part of the near plane to be proportional to the dimensions of the rendered image, the aspect ratio (defined as width divided by height) of the rendered image is the final value. The **aspect ratio** is used to specify the shape of the frustum, illustrated in Figure 3, which depicts the frustum as viewed from the front (along the negative z-axis).

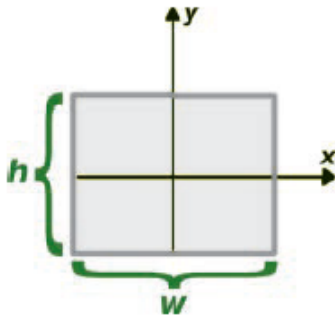


Figure 3. The aspect ratio $r = w/h$ of the frustum

In a perspective projection, points in space (within the frustum) are mapped to points in the **projection window**: a flat rectangular region in space corresponding to the rendered image that will be displayed on the computer screen. The projection window corresponds to the smaller rectangular side of the frustum, the side nearest to the origin. To visualize how a point P is transformed in a perspective projection, draw a line from P to the origin. The intersection of the line with the projection window is the result. Figure 4 illustrates the results of projecting three (3) different points within the frustum onto the projection window.

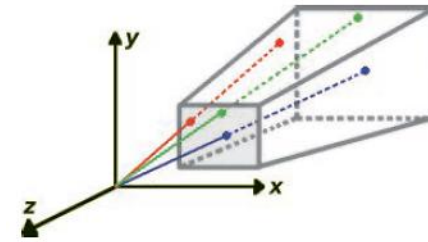


Figure 4. Projecting points from the frustum to the projection window

References:

- Korites, B. (2018). *Python graphics: A reference for creating 2D and 3D images*. Apress.
- Marschner, S. & Shirley, P. (2021). *Fundamentals of computer graphics* (5th ed.). CRC Press.
- Stemkoski, L. & Pascale, M. (2021). *Developing graphics frameworks with Python and OpenGL*. CRC Press.